



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 8

Multithreading

How to run code concurrently with threads, keep shared data safe from race conditions, and manage work efficiently with thread pools and the executor framework.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

8.1	Processes & Threads	Why concurrency?
8.2	Creating Threads	Thread vs Runnable
8.3	Thread Lifecycle & Control	start, sleep, join, states
8.4	Race Conditions	The danger of shared state
8.5	synchronized	Locks & mutual exclusion
8.6	volatile & Atomics	Visibility & lock-free counters
8.7	Coordination & Deadlock	wait/notify, deadlock
8.8	Executors & Thread Pools	ExecutorService, Future
8.9	Concurrent Collections	Thread-safe data structures
8.10	Common Mistakes & Tips	Pitfalls every beginner hits

Every program you've written so far ran on a **single thread** — one line executed, then the next, top to bottom. **Multithreading** lets a program do several things at once: download a file *while* updating a progress bar, or serve many users at the same time.

Process vs Thread

FEATURE	PROCESS	THREAD
What it is	A running program (its own JVM instance)	A path of execution <i>inside</i> a process
Memory	Own separate memory space	Shares memory with other threads in the same process
Cost to create	Heavy	Lightweight
Communication	Complex (IPC, sockets, files)	Easy — shared variables (but dangerous!)

Because threads **share memory**, they can cooperate cheaply — but that same sharing is the source of nearly every bug in this chapter. Keep it in mind: **shared, mutable state is the enemy**.

Concurrency vs Parallelism

- **Concurrency** — dealing with many tasks at once by rapidly switching between them (even on one CPU core).
- **Parallelism** — literally running tasks at the same instant on multiple CPU cores.

The main Thread

Your program already has one thread — the JVM starts `main()` on a thread named `main`. You can ask any thread for its name:

JAVA

```
1 public class Main {
2     public static void main(String[] args) {
3         Thread current = Thread.currentThread();
4         System.out.println("Running on thread: " + current.getName());
5     }
6 }
```

OUTPUT

```
Running on thread: main
```

NOTE

Modern hardware has multiple cores, so multithreading is how you actually *use* the machine. But threads also help even on a single core: while one thread waits for slow I/O (disk, network), another can keep working instead of the whole program freezing.

There are two classic ways to create a thread: **extend** `Thread`, or **implement** `Runnable`. Implementing `Runnable` is preferred — it keeps your class free to extend something else, and it separates the *task* from the *thread* that runs it.

Way 1 — Extend Thread

```

1  public class Worker extends Thread {
2      @Override
3      public void run() {
4          for (int i = 1; i ≤ 3; i++) {
5              System.out.println(getName() + " counting: " + i);
6          }
7      }
8  }
9
10 // Using it
11 Worker w = new Worker();
12 w.start();    // starts a NEW thread, which calls run()
```

JAVA

Way 2 — Implement Runnable (preferred)

```

1  public class Task implements Runnable {
2      @Override
3      public void run() {
4          System.out.println("Task running on " + Thread.currentThread().getName());
5      }
6  }
7
8  // Wrap the task in a Thread and start it
9  Task task = new Task();
10 Thread t = new Thread(task);
11 t.start();
```

JAVA

Way 3 — Runnable as a Lambda

Since `Runnable` is a functional interface (Chapter 7!), the cleanest form is a lambda:

```

1  Thread t = new Thread(() → {
2      System.out.println("Hello from " + Thread.currentThread().getName());
3  });
4  t.start();
5
6  // Even shorter for a quick task
7  new Thread(() → System.out.println("Quick job")).start();
```

JAVA

The #1 beginner error: calling `run()` instead of `start()`. `t.run()` just executes the method **on the current thread** — no new thread is created. Only `t.start()` spawns a new thread and then calls `run()` on it.

```
1 Thread t = new Thread(() → System.out.println(Thread.currentThread().getName()));
2
3 t.run();    // prints "main"          - WRONG, runs on the caller's thread
4 t.start();  // prints "Thread-0"     - CORRECT, runs on a new thread
```

Threads Run Out of Order

```
1 for (int i = 1; i ≤ 3; i++) {
2     final int id = i;
3     new Thread(() → System.out.println("Thread " + id + " ran")).start();
4 }
5 // Possible output - order is NOT guaranteed:
6 // Thread 2 ran
7 // Thread 1 ran
8 // Thread 3 ran
```

WARNING

You cannot predict the order in which threads execute, or how their output interleaves. The scheduler decides. **Never write code that assumes a particular ordering** unless you have explicitly coordinated it (with `join`, locks, etc.).

The Six Thread States

STATE	MEANING
NEW	Created but <code>start()</code> not yet called
RUNNABLE	Running or ready to run (waiting for CPU time)
BLOCKED	Waiting to acquire a lock held by another thread
WAITING	Waiting indefinitely (e.g. <code>join()</code> , <code>wait()</code>)
TIMED_WAITING	Waiting for a set time (e.g. <code>sleep(1000)</code>)
TERMINATED	Finished — <code>run()</code> has returned

sleep — pause the current thread

```
1 System.out.println("Start");
2 try {
3     Thread.sleep(1000);    // pause 1000 ms (1 second)
4 } catch (InterruptedException e) {
5     System.out.println("Sleep was interrupted");
6 }
7 System.out.println("...one second later");
```

JAVA

`sleep` is a `static` method — it always pauses the thread that *calls* it, and it throws the checked `InterruptedException`, so it must be handled.

join — wait for another thread to finish

Without `join`, the main thread races ahead and may finish before the workers. `join()` makes the caller wait until that thread terminates:

```
1 Thread worker = new Thread(() -> {
2     for (int i = 1; i ≤ 3; i++) {
3         System.out.println("working... " + i);
4     }
5 });
6
7 worker.start();
8 worker.join();    // main thread BLOCKS here until worker is done
9 System.out.println("Worker finished — now main continues");
```

JAVA

```
working... 1 working... 2 working... 3 Worker finished — now main continues
```

OUTPUT

Joining Multiple Threads

```

1  List<Thread> threads = new ArrayList<>();
2  for (int i = 0; i < 3; i++) {
3      Thread t = new Thread(() → System.out.println("done"));
4      t.start();
5      threads.add(t);
6  }
7  // Wait for ALL of them to complete
8  for (Thread t : threads) {
9      t.join();
10 }
11 System.out.println("All workers finished");

```

Other Useful Methods

METHOD	EFFECT
<code>Thread.sleep(ms)</code>	Pause the current thread for <code>ms</code> milliseconds
<code>t.join()</code>	Wait for thread <code>t</code> to finish
<code>t.interrupt()</code>	Request that thread <code>t</code> stop what it's doing
<code>t.isAlive()</code>	Has the thread started but not yet terminated?
<code>t.setDaemon(true)</code>	Mark as a daemon — JVM won't wait for it to exit
<code>t.setName(...)</code> / <code>getName()</code>	Set or read the thread's name

TIP

Java has no safe `stop()` method — it was deprecated because forcibly killing a thread can leave shared data half-updated. The right way to stop a thread is **cooperative**: call `interrupt()` and have the thread check `Thread.currentThread().isInterrupted()` and return on its own.

A **race condition** happens when two or more threads access shared data at the same time, and the final result depends on the unpredictable *timing* of their execution. This is the central problem of concurrency.

A Broken Counter

```
1 public class Counter {
2     private int count = 0;
3     public void increment() { count++; } // looks atomic - it is NOT
4     public int getCount() { return count; }
5 }
```

JAVA

```
1 Counter counter = new Counter();
2 List<Thread> threads = new ArrayList<>();
3
4 // 4 threads, each incrementing 1000 times → we expect 4000
5 for (int i = 0; i < 4; i++) {
6     Thread t = new Thread(() → {
7         for (int j = 0; j < 1000; j++) counter.increment();
8     });
9     t.start();
10    threads.add(t);
11 }
12 for (Thread t : threads) t.join();
13
14 System.out.println("Final count: " + counter.getCount());
```

JAVA

```
Final count: 3726 // WRONG! (and different every run)
```

OUTPUT

Why It Breaks

The innocent-looking `count++` is actually **three separate steps**: read the value, add one, write it back. Two threads can interleave and clobber each other:

```
1 // count starts at 10. Two threads run count++ "at the same time":
2
3 Thread A: read count (10)
4 Thread B: read count (10) // B read before A wrote back!
5 Thread A: add 1 → 11
6 Thread B: add 1 → 11
7 Thread A: write 11
8 Thread B: write 11 // one increment was LOST
9
10 // Result is 11, but it should be 12
```

JAVA

Race conditions are **intermittent** — the code may work 99 times and fail on the 100th. They often disappear when you add print statements or run a debugger (which changes the timing). This is exactly what makes them so dangerous. The fix is never "run it again" — it's proper synchronization.

The Three Ingredients of a Race

1. **Shared state** — data more than one thread can touch.
2. **Mutation** — at least one thread *modifies* it.
3. **No coordination** — nothing forces the accesses to happen one at a time.

Remove any one ingredient and the race is gone. The rest of this chapter is about removing ingredient #3.

The `synchronized` keyword enforces **mutual exclusion**: only one thread at a time can hold an object's **lock** (also called its *monitor*). Other threads that reach the synchronized code must wait their turn.

Synchronized Method

```

1  public class Counter {
2      private int count = 0;
3
4      // Only one thread can be inside increment() at a time
5      public synchronized void increment() { count++; }
6      public synchronized int getCount()   { return count; }
7  }

```

JAVA

Now the broken counter from 8.4 prints `4000` — every single run. The lock guarantees each `count++` completes fully before another thread starts one.

```
Final count: 4000
```

OUTPUT

Synchronized Block

Locking an entire method can be more than you need. A **synchronized block** locks only the critical section, so threads spend less time waiting:

```

1  public class BankAccount {
2      private int balance = 0;
3      private final Object lock = new Object(); // a dedicated lock object
4
5      public void deposit(int amount) {
6          // ... do non-critical work freely here ...
7          synchronized (lock) {
8              balance += amount; // only THIS is protected
9          }
10     }
11 }

```

JAVA

NOTE

A `synchronized` method locks on `this` (or the `Class` object for static methods). A synchronized block lets you pick the lock. Using a **private final lock object** is safer than locking on `this`, because outside code can't accidentally grab the same lock and interfere.

The reentrant Lock Class

For more control, `java.util.concurrent.locks.ReentrantLock` offers explicit lock/unlock, the ability to try-lock, and fairness options. Always unlock in a `finally` block:

```
1  import java.util.concurrent.locks.ReentrantLock;
2
3  public class Counter {
4      private int count = 0;
5      private final ReentrantLock lock = new ReentrantLock();
6
7      public void increment() {
8          lock.lock();
9          try {
10              count++;
11          } finally {
12              lock.unlock();    // ALWAYS unlock, even if an exception is thrown
13          }
14      }
15  }
```

TIP

Keep synchronized sections **as small as possible**. The more code you hold a lock for, the more other threads sit idle waiting — which throws away the performance you wanted from threading in the first place. Lock only the shared data access, nothing more.

The Visibility Problem

Locks solve interleaving, but there's a second, subtler issue: **visibility**. Each thread may cache a copy of a variable, so one thread's write might never be seen by another. This loop can run forever even after `running` is set to false:

```
1 public class Task {
2     private boolean running = true;    // BUG: not volatile
3
4     public void run() {
5         while (running) { /* work */ }    // may never see running = false!
6     }
7     public void stop() { running = false; }
8 }
```

JAVA

volatile — guarantee visibility

Marking a field `volatile` forces every read and write to go straight to main memory, so all threads always see the latest value:

```
1 public class Task {
2     private volatile boolean running = true;    // FIXED
3
4     public void run() {
5         while (running) { /* work */ }    // now sees the update promptly
6     }
7     public void stop() { running = false; }
8 }
```

JAVA

WARNING

`volatile` guarantees **visibility**, but NOT **atomicity**. `volatile int count; count++;` is still a race, because `count++` is three operations. Use `volatile` only for simple flags where a single thread writes and others read. For compound updates, use synchronization or atomics.

Atomic Variables — lock-free safety

The `java.util.concurrent.atomic` package provides classes like `AtomicInteger` whose operations are both atomic *and* visible — without any explicit locking:

```
1 import java.util.concurrent.atomic.AtomicInteger;
2
3 public class Counter {
4     private final AtomicInteger count = new AtomicInteger(0);
5
6     public void increment() { count.incrementAndGet(); }    // atomic ++
7     public int getCount() { return count.get(); }
8 }
```

JAVA

wait / notify — threads signalling each other

Sometimes a thread must **wait** until some condition becomes true — a producer waiting for space, a consumer waiting for data. The `wait()` / `notify()` methods (called inside a synchronized block) let threads sleep and wake each other:

```

1  class Message {
2      private String content;
3      private boolean available = false;
4
5      public synchronized void put(String text) {
6          content = text;
7          available = true;
8          notify();    // wake a thread waiting in take()
9      }
10
11     public synchronized String take() throws InterruptedException {
12         while (!available) {
13             wait();    // release the lock and sleep until notified
14         }
15         available = false;
16         return content;
17     }
18 }

```

JAVA

NOTE

Always call `wait()` inside a `while` loop that re-checks the condition, never a single `if`. A thread can wake up without the condition being true (a "spurious wakeup"), so it must verify and go back to waiting. `wait()`, `notify()`, and `notifyAll()` can only be called while holding that object's lock.

Deadlock — everyone waits forever

A **deadlock** occurs when two threads each hold a lock the other needs, so neither can ever proceed:

```

1  Object lockA = new Object();
2  Object lockB = new Object();
3
4  // Thread 1: grabs A, then wants B
5  new Thread(() → {
6      synchronized (lockA) {
7          synchronized (lockB) { /* ... */ }
8      }
9  }).start();
10
11 // Thread 2: grabs B, then wants A ← opposite order!
12 new Thread(() → {
13     synchronized (lockB) {
14         synchronized (lockA) { /* ... */ }
15     }
16 }).start();
17 // If both grab their first lock at once, they wait on each other FOREVER

```

JAVA

The classic cause of deadlock is acquiring multiple locks in **different orders** in different threads. The simplest cure: always acquire locks in the **same global order** everywhere. If every thread grabs `LockA` before `LockB`, this deadlock cannot happen.

Avoiding Deadlock

- Acquire multiple locks in a **consistent order** across all threads.
- Hold locks for the **shortest time** possible; avoid nesting them.
- Prefer `tryLock(timeout)` on `ReentrantLock` so a thread can give up instead of blocking forever.
- Best of all — use the higher-level tools in the next section and avoid manual locking entirely.

Manually creating a `new Thread()` for every task is wasteful — thread creation is expensive, and thousands of threads will exhaust memory. The **executor framework** (`java.util.concurrent`) manages a reusable **pool of threads** for you. This is how real applications do concurrency.

Submitting Tasks to a Pool

```

1  import java.util.concurrent.ExecutorService;
2  import java.util.concurrent.Executors;
3
4  // A pool of 3 reusable threads
5  ExecutorService pool = Executors.newFixedThreadPool(3);
6
7  for (int i = 1; i ≤ 6; i++) {
8      final int taskId = i;
9      pool.submit(() → {
10         System.out.println("Task " + taskId + " on " +
11             Thread.currentThread().getName());
12     });
13 }
14
15 pool.shutdown(); // no new tasks accepted; existing ones finish

```

JAVA

Task 1 on pool-1-thread-1 Task 2 on pool-1-thread-2 Task 3 on pool-1-thread-3 Task 4 on pool-1-thread-1 Task 5 on ^{OUTPUT} pool-1-thread-2 Task 6 on pool-1-thread-3

Notice only **three** threads handle all six tasks — they're reused. That's the whole point of a pool.

Kinds of Pools

FACTORY METHOD

WHAT YOU GET

`newFixedThreadPool(n)`

Exactly `n` threads, reused for all tasks

`newCachedThreadPool()`

Creates threads as needed, reuses idle ones

`newSingleThreadExecutor()`

One thread — tasks run in order, one at a time

`newScheduledThreadPool(n)`

For delayed or periodic tasks

Callable and Future — getting a result back

A `Runnable` returns nothing. A `Callable<T>` returns a value and may throw checked exceptions. `submit` hands you a `Future<T>` — a placeholder for a result that will arrive later:


```

1  import java.util.concurrent.*;
2
3  ExecutorService pool = Executors.newFixedThreadPool(2);
4
5  Callable<Integer> task = () -> {
6      int sum = 0;
7      for (int i = 1; i ≤ 100; i++) sum += i;
8      return sum;
9  };
10
11 Future<Integer> future = pool.submit(task);
12
13 // ... do other work while the task runs ...
14
15 Integer result = future.get(); // BLOCKS until the result is ready
16 System.out.println("Sum = " + result); // Sum = 5050
17
18 pool.shutdown();

```

Sum = 5050

OUTPUT

Waiting for a Whole Pool to Finish

JAVA

```

1  pool.shutdown();
2  // Block until all tasks finish, or the timeout elapses
3  boolean done = pool.awaitTermination(10, TimeUnit.SECONDS);
4  if (done) {
5      System.out.println("All tasks completed");
6  }

```

WARNING

Always call `shutdown()` when you're finished with an `ExecutorService`. Its threads are non-daemon by default, so forgetting to shut it down keeps the JVM alive and your program will **never exit**. Also note `future.get()` blocks the calling thread until the result is ready.

TIP

Prefer the executor framework over raw `Thread` objects for almost all real work. It reuses threads, bounds resource usage, gives you results via `Future`, and handles the lifecycle. Reach for `new Thread()` only for quick experiments or a single background job.

The ordinary collections from Chapter 5 — `ArrayList`, `HashMap`, `HashSet` — are **not thread-safe**. If several threads modify one at the same time, you can corrupt it or get a `ConcurrentModificationException`. The `java.util.concurrent` package provides safe replacements.

The Unsafe Way

```

1  // DANGER - HashMap shared across threads without protection
2  Map<String, Integer> map = new HashMap<>();
3
4  Runnable job = () -> {
5      for (int i = 0; i < 1000; i++) {
6          map.put(Thread.currentThread().getName() + i, i);
7      }
8  };
9  // Running this on several threads can corrupt the map's internal structure

```

JAVA

ConcurrentHashMap — the safe map

```

1  import java.util.concurrent.ConcurrentHashMap;
2
3  Map<String, Integer> safeMap = new ConcurrentHashMap<>();
4
5  // Safe to call from many threads at once
6  safeMap.put("a", 1);
7  safeMap.merge("a", 1, Integer::sum); // atomic "add to existing value"
8  System.out.println(safeMap.get("a")); // 2

```

JAVA

The Main Concurrent Collections

INSTEAD OF...	USE...	NOTES
<code>HashMap</code>	<code>ConcurrentHashMap</code>	High-performance thread-safe map
<code>ArrayList</code>	<code>CopyOnWriteArrayList</code>	Great when reads vastly outnumber writes
<code>HashSet</code>	<code>ConcurrentHashMap.newKeySet()</code>	Thread-safe set
a producer/consumer queue	<code>BlockingQueue</code>	Threads block until an item is available / space frees

BlockingQueue — producer / consumer made easy

A `BlockingQueue` handles all the `wait` / `notify` logic from 8.7 for you. Producers `put`, consumers `take`, and the queue blocks automatically:

```
1 import java.util.concurrent.*;
2
3 BlockingQueue<String> queue = new LinkedBlockingQueue<>();
4
5 // Producer
6 new Thread(() → {
7     try {
8         queue.put("order-1");    // blocks if the queue is full
9         queue.put("order-2");
10    } catch (InterruptedException e) { }
11 }).start();
12
13 // Consumer
14 new Thread(() → {
15     try {
16         System.out.println("Got: " + queue.take());    // blocks if empty
17         System.out.println("Got: " + queue.take());
18    } catch (InterruptedException e) { }
19 }).start();
```

TIP

If you only need a quick thread-safe wrapper around an existing collection, `Collections.synchronizedList(list)` works — but it locks the whole collection on every access. The dedicated concurrent collections scale far better under real contention and should be your default for shared data.

1. Calling run() Instead of start()

```
1 Thread t = new Thread(() → System.out.println(Thread.currentThread().getName()));
2
3 t.run();    // WRONG – runs on the current thread, no concurrency ("main")
4 t.start();  // CORRECT – spawns a new thread ("Thread-0")
```

JAVA

2. Assuming an Execution Order

```
1 // WRONG – output order is undefined; this may print in any order
2 new Thread(() → System.out.println("A")).start();
3 new Thread(() → System.out.println("B")).start();
4
5 // If you truly need order, coordinate explicitly with join()
6 Thread a = new Thread(() → System.out.println("A"));
7 a.start();
8 a.join();    // wait for A before starting B
9 new Thread(() → System.out.println("B")).start();
```

JAVA

3. Unprotected Shared Mutable State

```
1 // WRONG – count++ is not atomic; result is unpredictable
2 private int count = 0;
3 public void increment() { count++; }
4
5 // CORRECT – pick ONE: synchronized, or an atomic type
6 public synchronized void increment() { count++; }
7 // or
8 private final AtomicInteger count = new AtomicInteger();
9 public void increment() { count.incrementAndGet(); }
```

JAVA

4. Forgetting to Shut Down an Executor

```
1 ExecutorService pool = Executors.newFixedThreadPool(4);
2 pool.submit(() → doWork());
3 // BUG – no shutdown(): the JVM never exits, program hangs forever
4
5 // CORRECT
6 pool.shutdown();
```

JAVA

5. Swallowing InterruptedException

```

1  // BAD – silently ignoring the interrupt request
2  try {
3      Thread.sleep(1000);
4  } catch (InterruptedException e) { }    // now nobody can stop this thread
5
6  // BETTER – restore the interrupt flag so callers know
7  try {
8      Thread.sleep(1000);
9  } catch (InterruptedException e) {
10     Thread.currentThread().interrupt();    // preserve the signal
11 }

```

6. Locking on a Changing or Shared Object

```

1  // BAD – locking on a value that may be reassigned, or on a boxed Integer
2  private Integer lock = 0;
3  synchronized (lock) { lock++; }    // lock now points to a DIFFERENT object!
4
5  // GOOD – a dedicated, final lock object that never changes
6  private final Object lock = new Object();
7  synchronized (lock) { /* ... */ }

```

WARNING

Concurrency bugs don't show up reliably in testing — they depend on timing and hardware. Code that "passes on my machine" can fail under load in production. Design for safety from the start: minimize shared mutable state, and prefer high-level tools (executors, atomics, concurrent collections) over hand-written locking.

QUICK REFERENCE – THREADING DECISIONS

- › Create tasks as `Runnable` / `Callable` lambdas; run them via an `ExecutorService`, not raw threads
- › Use `start()`, never `run()`, to launch a thread
- › Use `join()` to wait for a thread; `awaitTermination()` to wait for a pool
- › Protect shared mutable data with `synchronized`, a `ReentrantLock`, or an atomic type
- › `volatile` = visibility only; not enough for compound updates like `x++`
- › Acquire multiple locks in a consistent order to avoid deadlock
- › Use `ConcurrentHashMap` / `BlockingQueue` instead of plain collections when sharing across threads
- › Always `shutdown()` an executor, and never swallow `InterruptedException`

QUICK REFERENCE – THE GOLDEN RULE

- › The safest concurrent code shares **nothing mutable** between threads
- › If you must share, make it **immutable**, or confine it to one thread
- › If it must be shared and mutable, **synchronize every access** — reads included
- › When possible, let the library (executors, atomics, concurrent collections) do the hard part

This document was compiled from the following resources. All content was adapted and restructured for educational use.

[1] **Oracle — Java Tutorials: Concurrency**

Threads, synchronization, liveness, guarded blocks, high-level concurrency objects

<https://docs.oracle.com/javase/tutorial/essential/concurrency/>

[2] **Programiz — Java Multithreading**

Creating threads, Thread vs Runnable, thread lifecycle, sleep & join

<https://www.programiz.com/java-programming/multithreading>

[3] **GeeksforGeeks — Multithreading in Java**

Thread states, synchronization, race conditions, deadlock, examples

<https://www.geeksforgeeks.org/multithreading-in-java/>

[4] **Baeldung — Guide to java.util.concurrent**

ExecutorService, Callable & Future, atomics, concurrent collections

<https://www.baeldung.com/java-util-concurrent>

[5] **Java Platform SE — java.util.concurrent — API documentation**

Executors, ExecutorService, Future, locks, atomic, ConcurrentHashMap, BlockingQueue

<https://docs.oracle.com/en/java/javase/17/docs/api/>

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com